

Vortex



Marc Muntané Clarà

Proyecto de IA local

Portfolio personal - DAW / DAM

Modelo local + API + RAG + agente + frontend React

Índice

1 - Definición del proyecto	3
1.1 - Presentación	3
1.2 - Descripción corta	3
1.3 - Objetivos	3
2 - Arquitectura y stack local	4
2.1 - Componentes principales	4
2.2 - Flujo de trabajo	4
3 - Interfaz frontend	5
3.1 - Chat principal	6
3.2 - Modo agente	7
3.3 - Detalles del chat	8
3.4 - Configuración general	9
3.5 - Apariencia	10
3.6 - Permisos	11
3.7 - Skills	12
3.8 - Perfiles	13
3.9 - Paleta de comandos	14
3.10 - Modo oscuro	15
4 - RAG y memoria local	16
5 - Agente local	17
6 - Backend y API	18
7 - Seguridad y privacidad	19
8 - Pruebas técnicas	20
9 - Encaje en el porfolio	21
10 - Conclusión	22

1 - Definición del proyecto

1.1 - Presentación

Vortex es mi IA local: una aplicación pensada para trabajar desde mi propio equipo, con una interfaz clara, memoria local y capacidad de actuar como asistente técnico dentro de un proyecto.

1.2 - Descripción corta

El objetivo principal es tener un entorno privado y controlado para preguntar, razonar sobre código, consultar conocimiento propio y ejecutar tareas de soporte sin depender siempre de servicios externos.

1.3 - Objetivos

- **Privacidad:** mantener conversaciones, memoria y contexto de proyecto en local.
- **Productividad:** reducir tiempo al revisar código, documentación y decisiones técnicas.
- **Control:** separar permisos de lectura, edición y acciones completas dentro del workspace.
- **Escalabilidad:** permitir perfiles, skills y mejoras del agente sin rehacer la interfaz.

2 - Arquitectura y stack local

2.1 - Componentes principales

Capa	Función	Valor dentro de Vortex
Frontend React	Interfaz, sesiones, configuración y modo agente.	Hace visible todo el flujo de trabajo.
API Python	Puente entre interfaz, modelo, RAG y acciones.	Centraliza llamadas y permisos.
LLM local	Motor de respuesta ejecutado en local.	Reduce dependencia externa.
RAG	Recuperación de memoria y documentos propios.	Da contexto real a las respuestas.
Agente	Análisis y tareas sobre el proyecto autorizado.	Convierte la IA en herramienta de trabajo.

2.2 - Flujo de trabajo

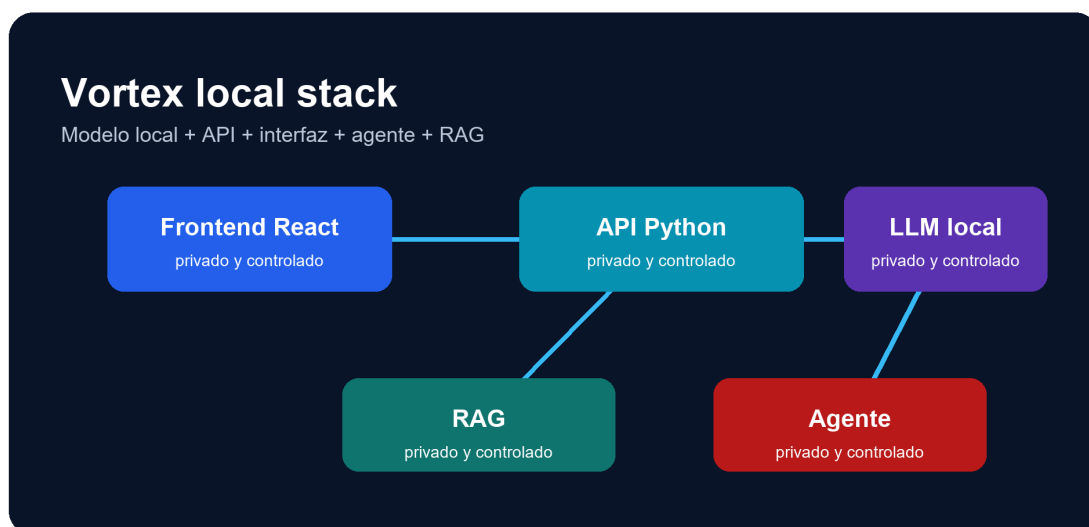


Diagrama general del stack local de Vortex.

3 - Interfaz frontend

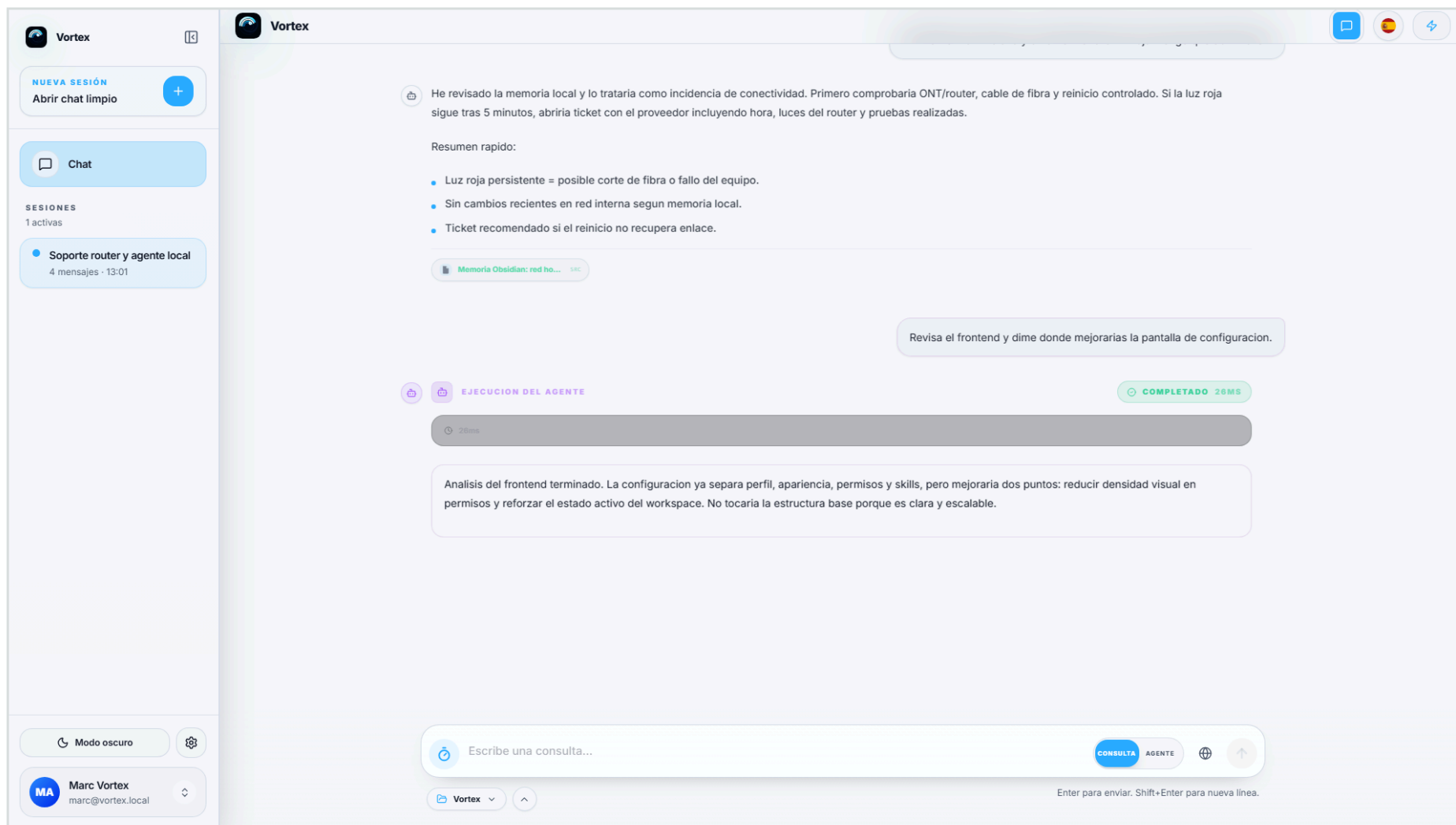
La interfaz es una parte importante del proyecto porque Vortex no solo responde texto: también organiza sesiones, muestra razonamiento, gestiona perfiles, separa permisos y permite cambiar entre consulta normal y agente.

Las capturas siguientes están hechas con la ventana completa del frontend. Se mantiene el contexto visual real: barra lateral, chat, configuración, modales y estados de la aplicación.

Estilo	Minimalista, claro, con estados visibles y separación entre acciones normales y modo agente.
Función	Centralizar chat local, workspace, permisos, skills, perfiles y preferencias.
Criterio visual	Capturas a pantalla completa para que se vea el producto real, no fragmentos aislados.

3.1 - Chat principal

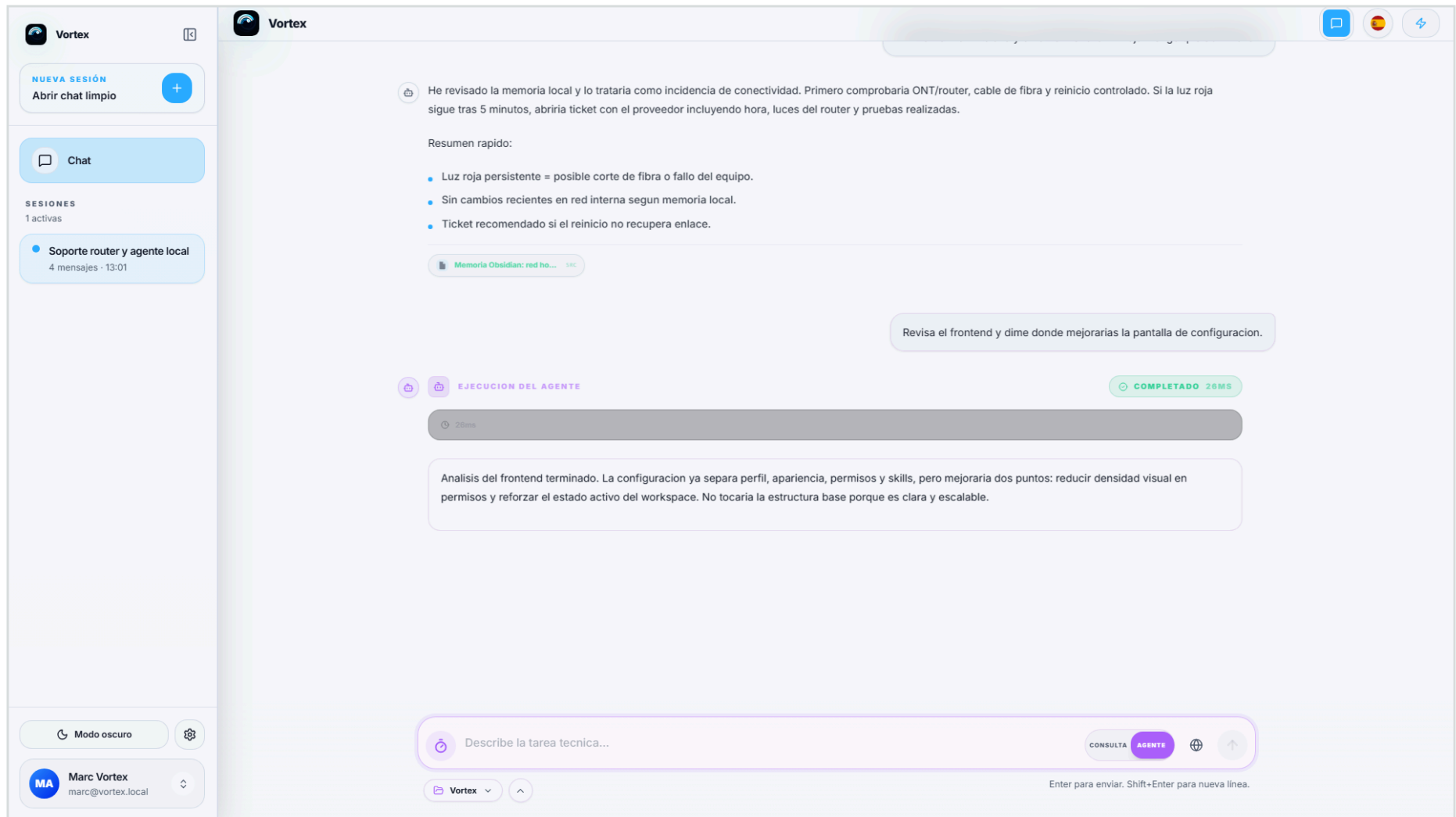
Vista completa del chat con una respuesta técnica y memoria local aplicada.



Captura fullscreen - 3.1: Chat principal.

3.2 - Modo agente

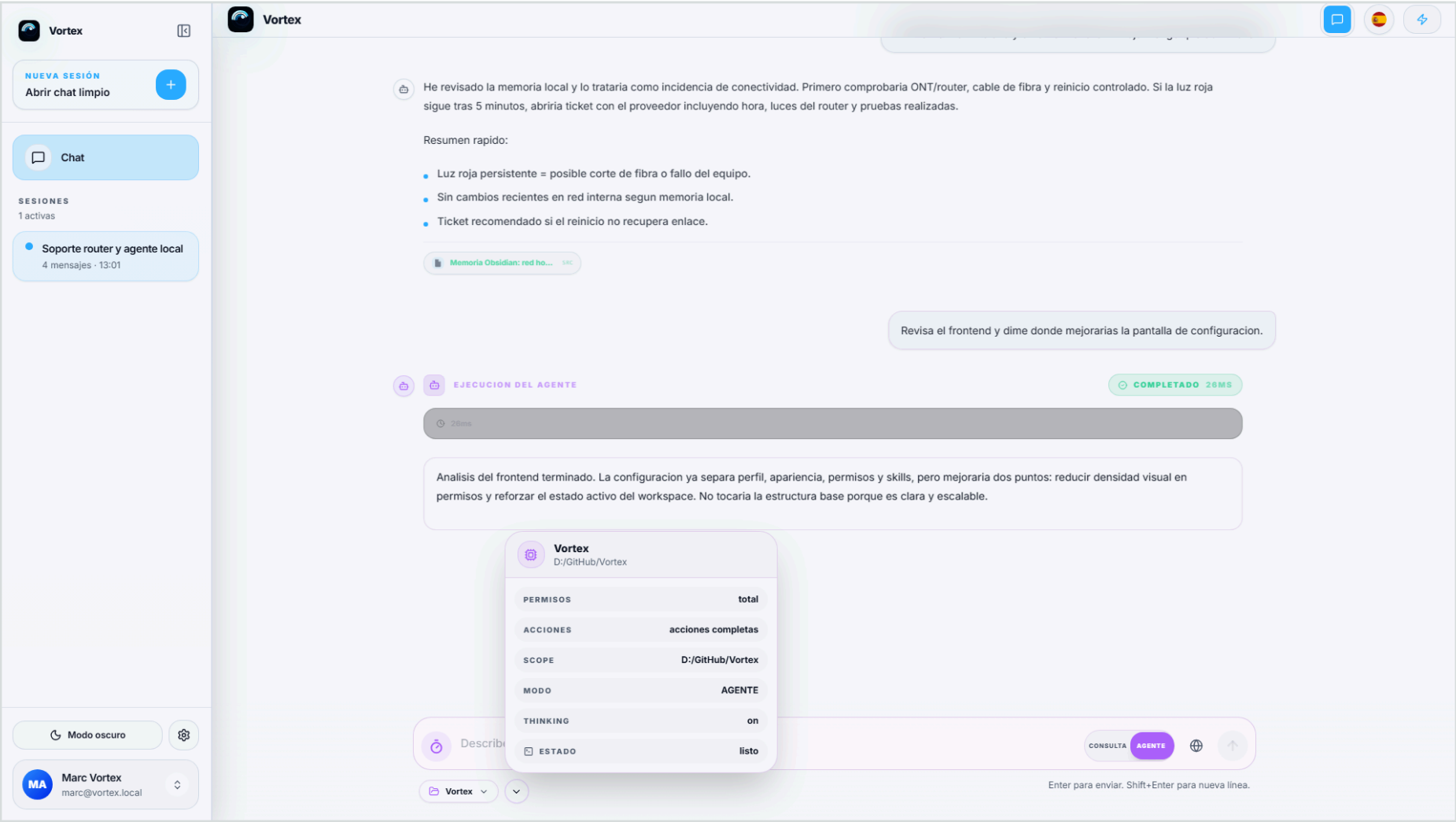
Selector de modo agente activo desde el compositor de mensajes.



Captura fullscreen - 3.2: Modo agente.

3.3 - Detalles del chat

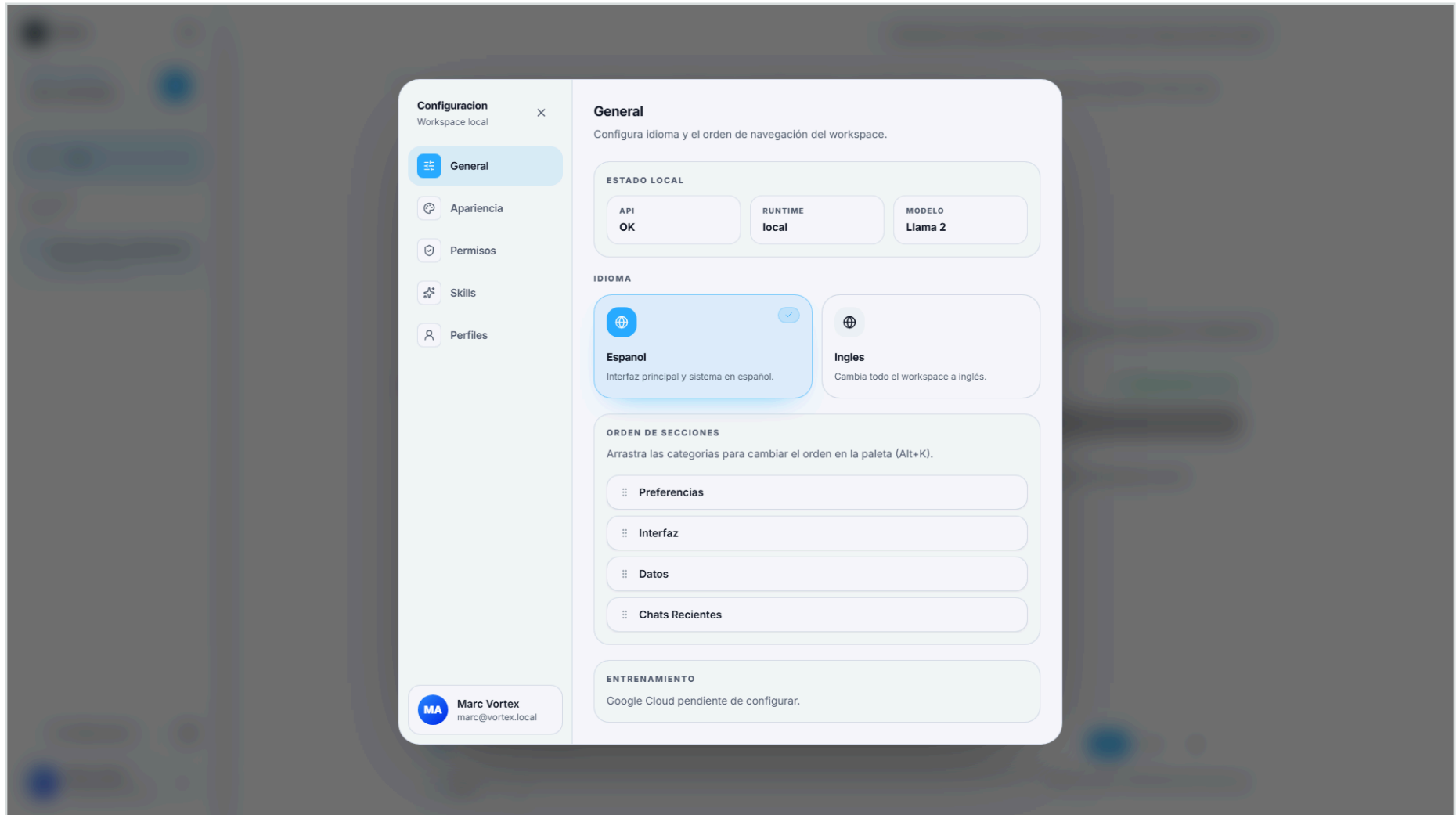
Panel de detalles con estado, permisos y metadatos de la conversación.



Captura fullscreen - 3.3: Detalles del chat.

3.4 - Configuración general

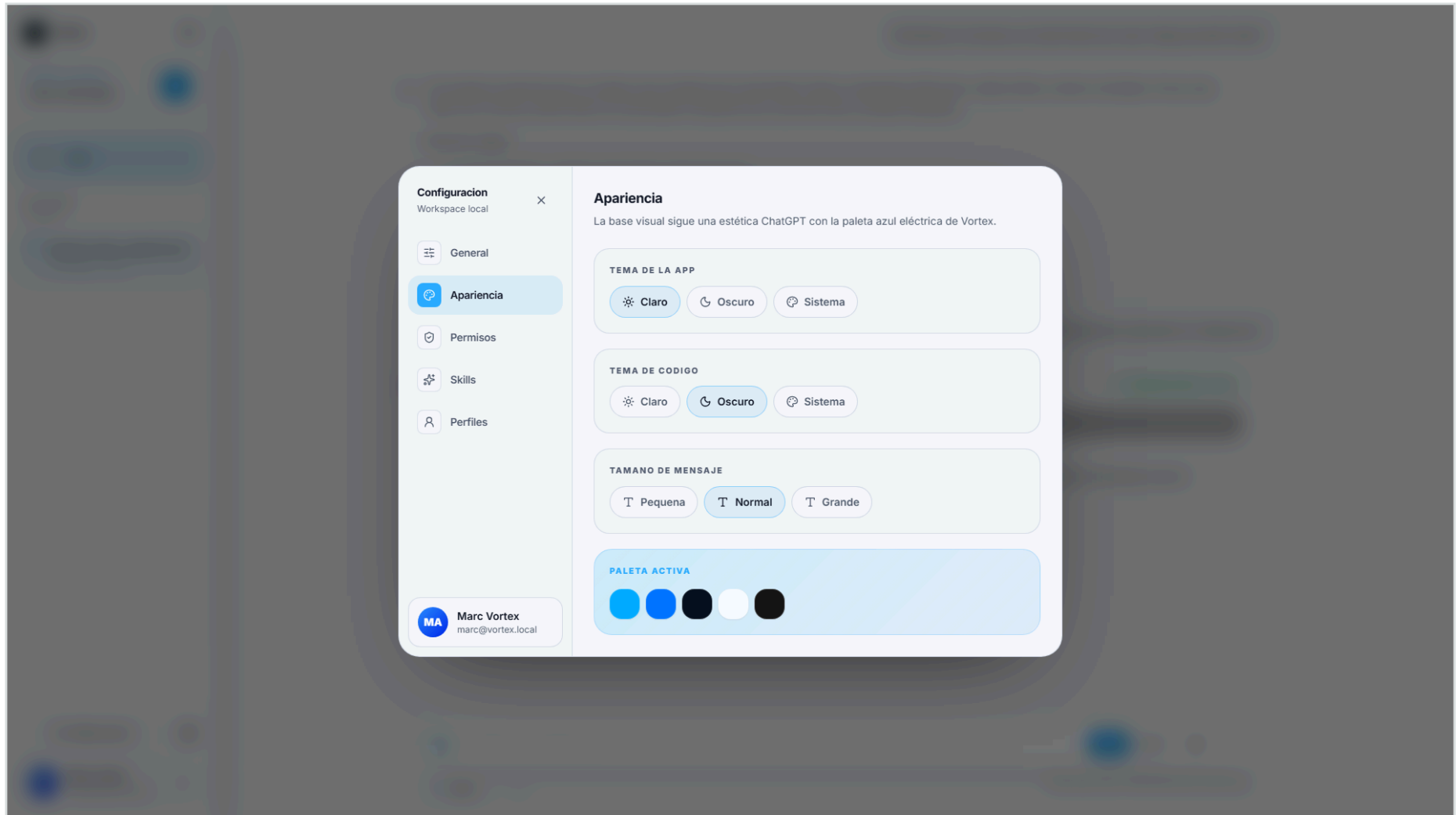
Ajustes generales: idioma, estado local, runtime y orden de secciones.



Captura fullscreen - 3.4: Configuración general.

3.5 - Apariencia

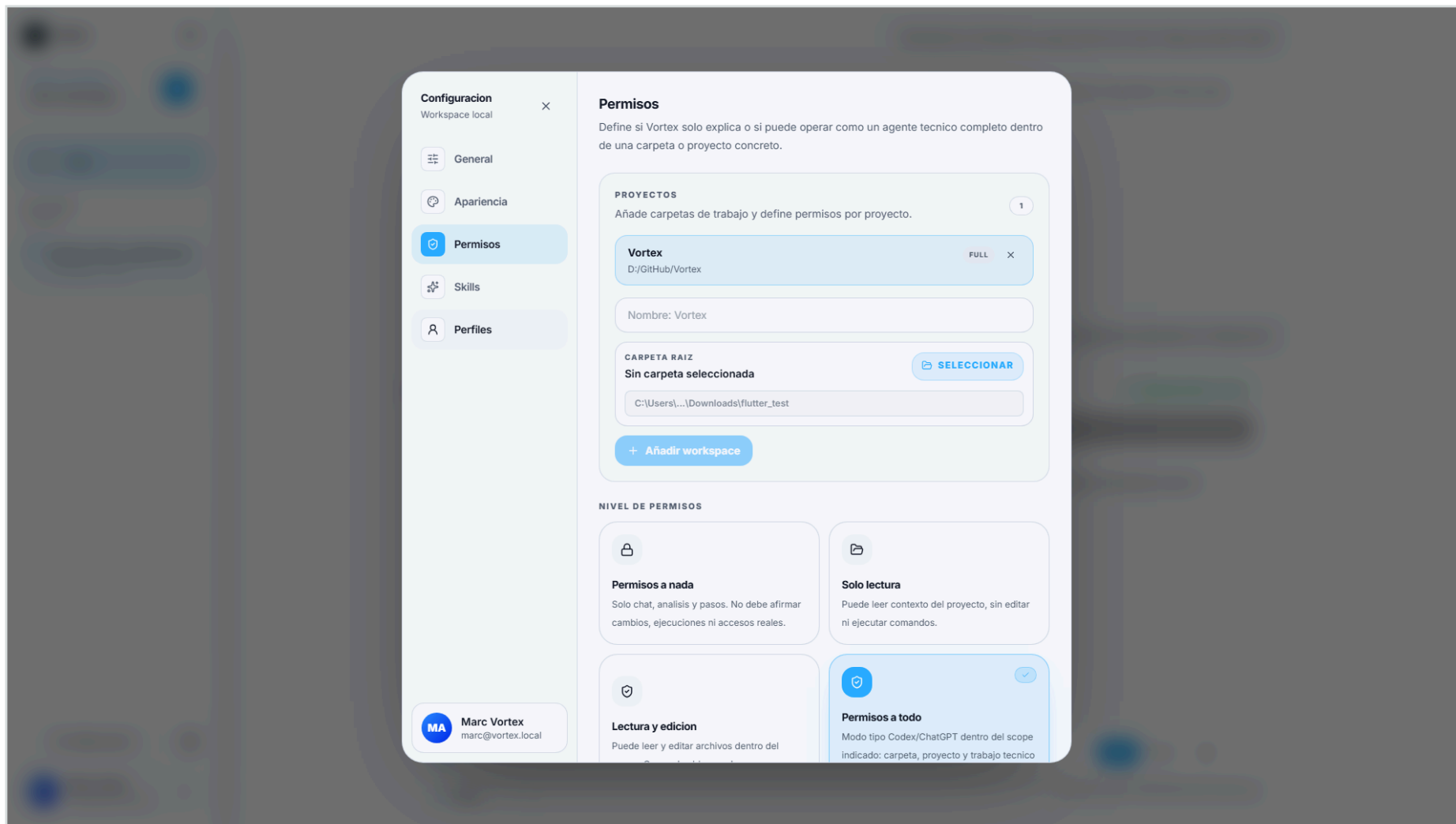
Opciones de tema, tamaño de mensaje, tema de código y paleta activa.



Captura fullscreen - 3.5: Apariencia.

3.6 - Permisos

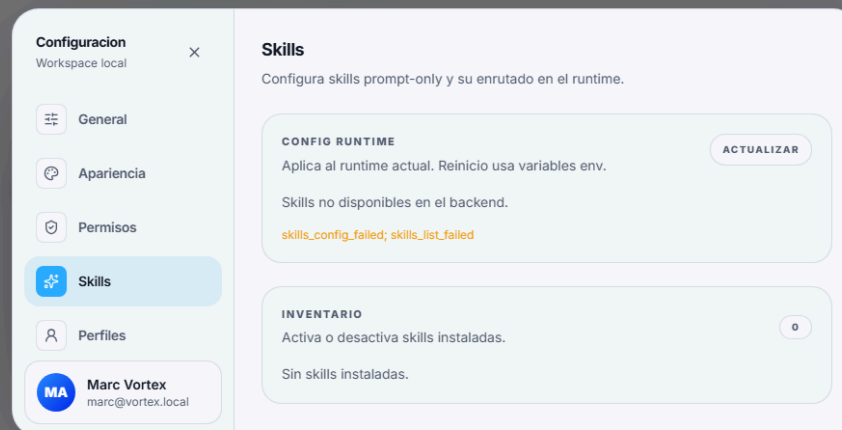
Gestión del workspace, alcance de permisos y acciones permitidas.



Captura fullscreen - 3.6: Permisos.

3.7 - Skills

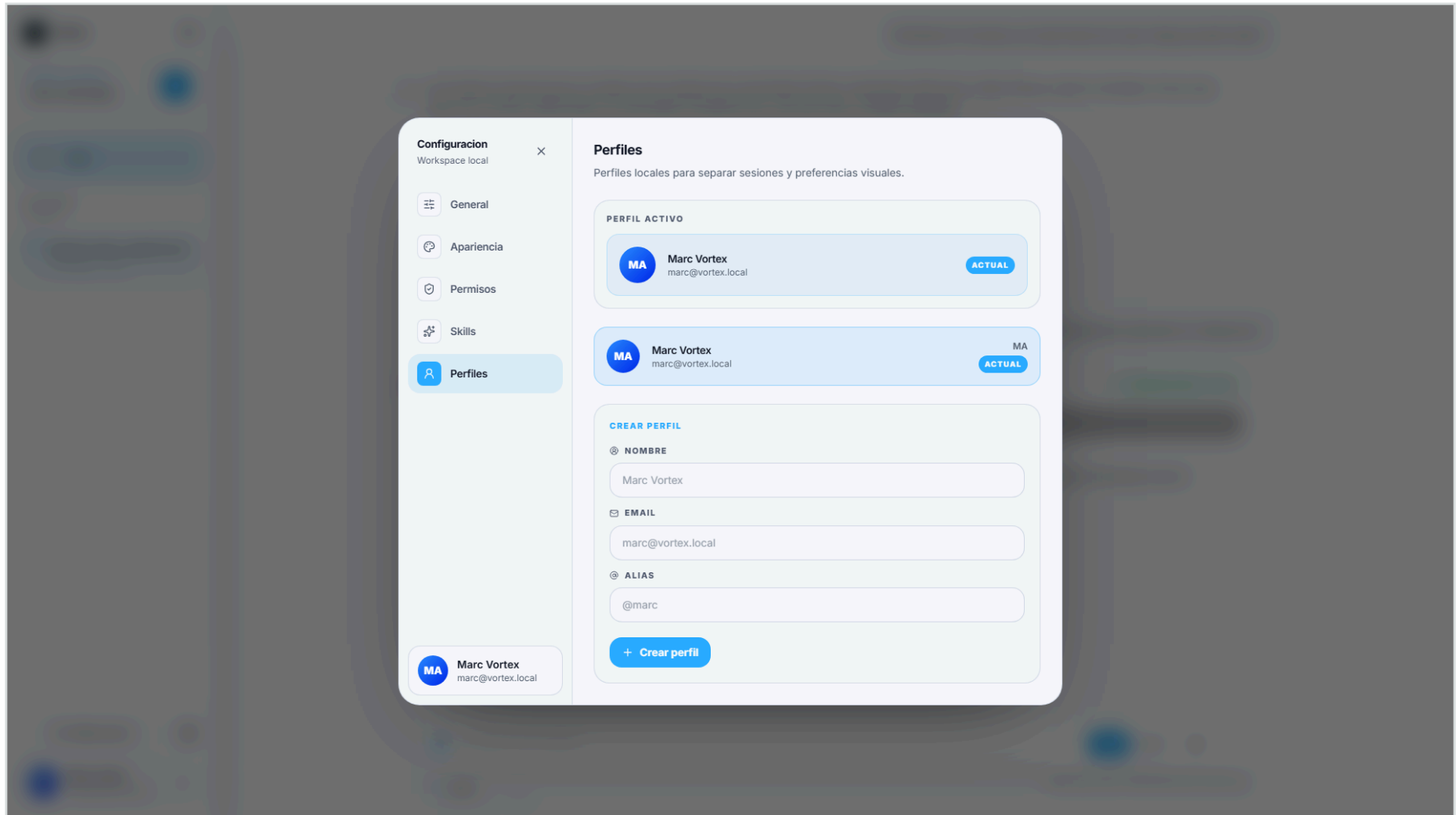
Inventario de skills locales y configuración runtime del RAG.



Captura fullscreen - 3.7: Skills.

3.8 - Perfiles

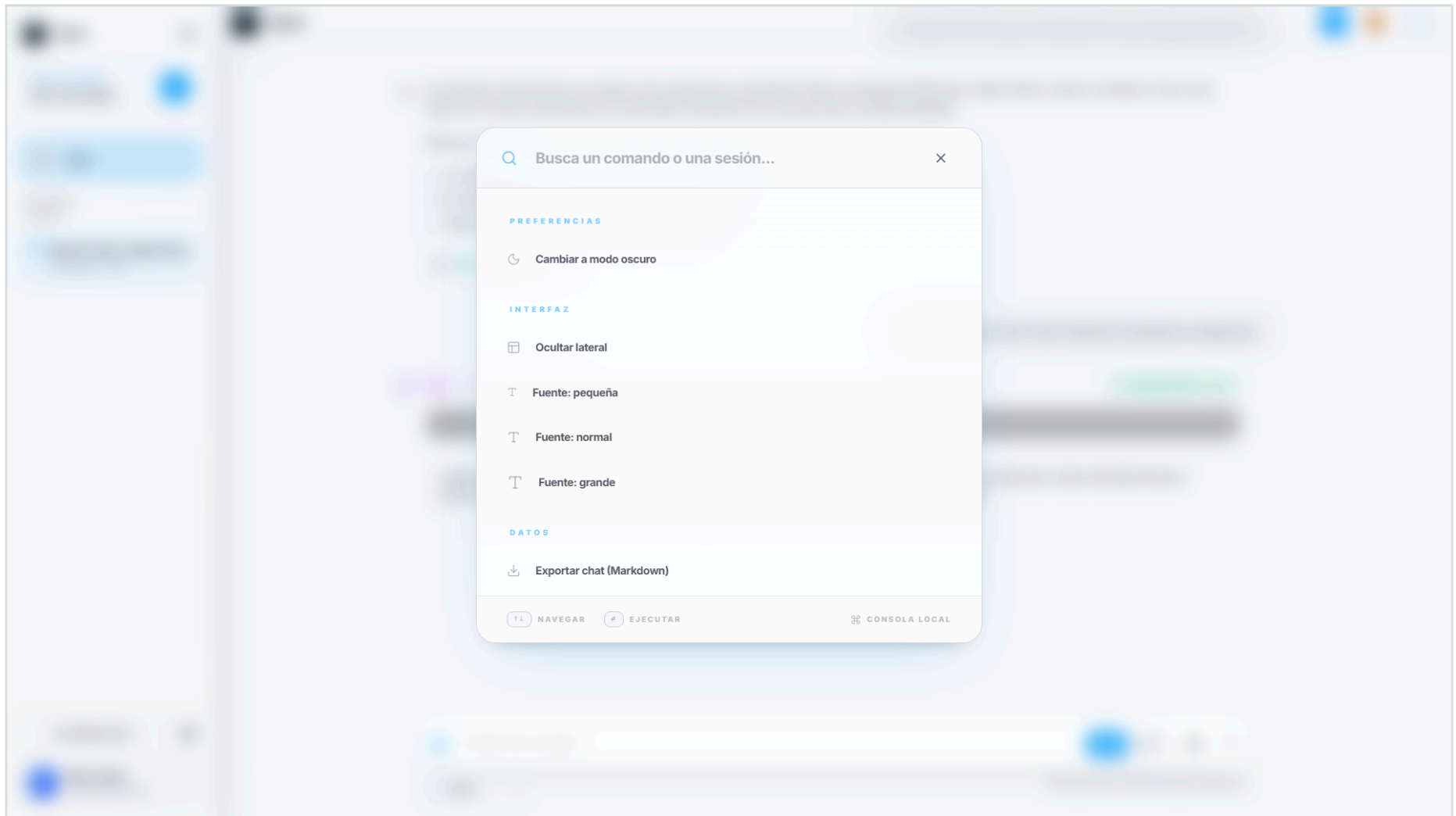
Perfiles locales para separar sesiones, preferencias y contexto.



Captura fullscreen - 3.8: Perfiles.

3.9 - Paleta de comandos

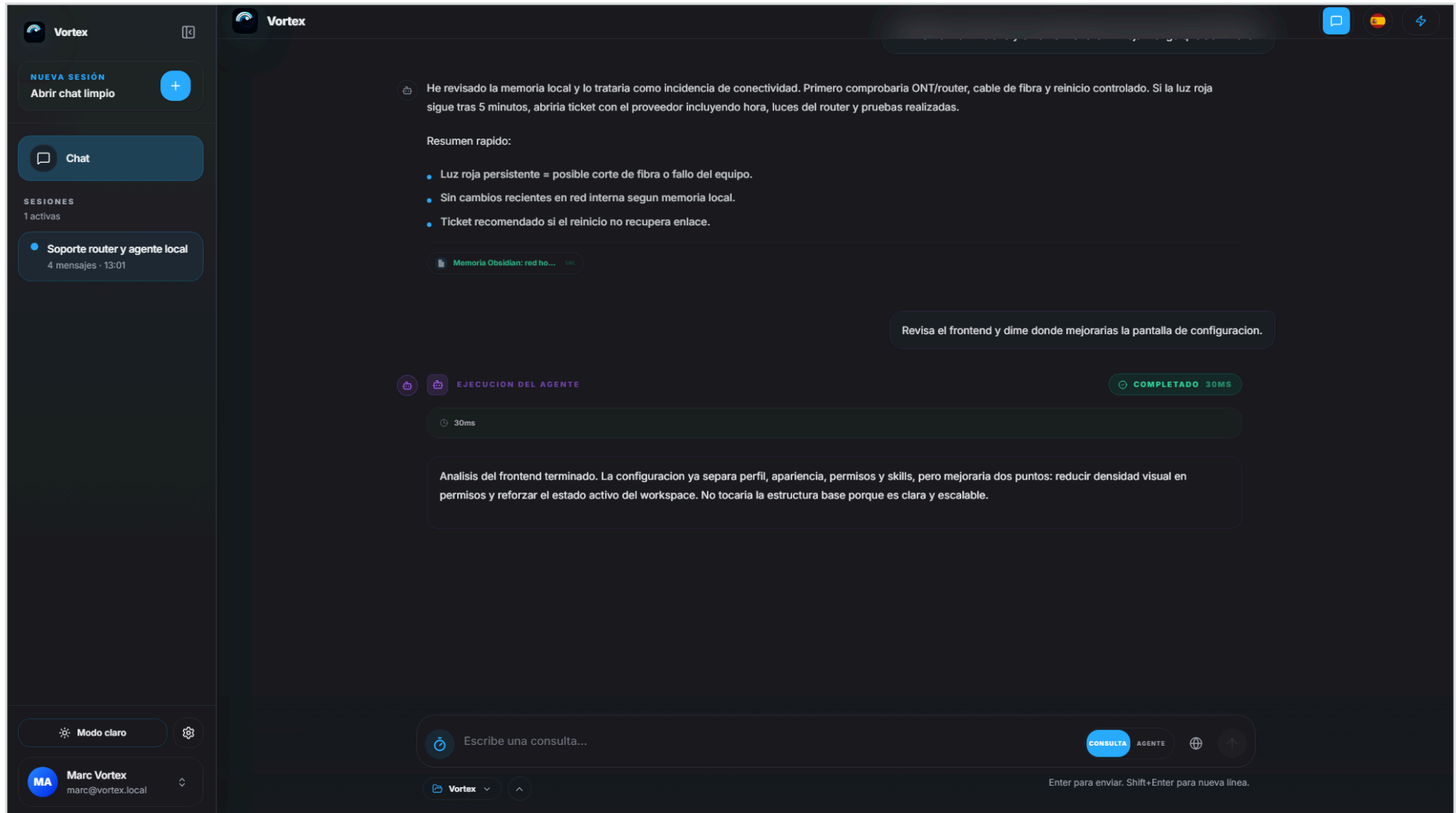
Acciones rápidas y preferencias accesibles por teclado.



Captura fullscreen - 3.9: Paleta de comandos.

3.10 - Modo oscuro

Interfaz en modo oscuro manteniendo contraste y estructura.



Captura fullscreen - 3.10: Modo oscuro.

4 - RAG y memoria local

El RAG permite que Vortex responda con información propia. En vez de tratar cada pregunta como algo aislado, puede recuperar notas, documentación y contexto del entorno local.

- **Fuentes locales:** memoria curada, documentos técnicos y referencias del proyecto.
- **Respuesta contextual:** la salida puede apoyarse en información real del usuario y no solo en el modelo base.
- **Control de coste:** la recuperación limita el contexto a lo importante para no saturar el prompt.

5 - Agente local

El modo agente convierte Vortex en una herramienta de trabajo sobre el proyecto. La idea no es solo conversar, sino analizar archivos, detectar problemas y preparar acciones bajo permisos definidos.

Modo	Qué permite	Control
Consulta	Responder, explicar y razonar.	No modifica archivos.
Agente seguro	Analizar proyecto y proponer cambios.	Acciones limitadas.
Agente completo	Trabajar dentro del workspace autorizado.	Depende de permisos explícitos.

6 - Backend y API

La API actúa como capa de coordinación. Separa el frontend del motor local, controla sesiones, sincroniza configuración y expone endpoints para skills, estado y validación del workspace.

- **Estado:** comprueba si modelo, backend y runtime están disponibles.
- **Sesiones:** mantiene conversaciones y perfiles locales separados.
- **Workspace:** valida proyectos autorizados antes de permitir acciones.
- **Skills:** configura recuperación, presupuesto de tokens y prioridad de herramientas.

7 - Seguridad y privacidad

La privacidad es una decisión de diseño, no un extra. Vortex prioriza ejecución local, permisos visibles y separación entre lectura, edición y acciones completas.

Permisos	Niveles separados: nada, solo lectura, lectura/edición y permisos completos.
Scope	El agente trabaja dentro del proyecto y carpeta autorizados.
Transparencia	La interfaz muestra estado, modo activo, workspace y acciones posibles.
Internet	No se activa por defecto; se trata como una capacidad explícita.

8 - Pruebas técnicas

Para validar Vortex, las pruebas se centran en que la aplicación sea útil en situaciones reales: respuesta de soporte, razonamiento sobre código, configuración de permisos y cambio de tema.

Prueba	Resultado esperado	Estado
Respuesta técnica	Detectar problema y dar pasos claros.	Correcto
Memoria local	Citar o usar contexto propio.	Correcto
Modo agente	Diferenciar consulta de ejecución.	Correcto
Permisos	Mostrar scope y nivel activo.	Correcto
Tema visual	Mantener legibilidad en claro y oscuro.	Correcto

9 - Encaje en el portfolio

Vortex encaja como proyecto de portfolio porque muestra una mezcla de frontend, backend, IA local, producto y criterio de seguridad. No es una demo aislada: es una herramienta pensada para uso propio.

- **Frontend:** interfaz cuidada, responsive y con configuración completa.
- **Backend:** API para sesiones, estado, skills y workspace.
- **IA:** modelo local, RAG, memoria y modo agente.
- **Producto:** perfiles, permisos y experiencia visual coherente.

10 - Conclusión



Vortex demuestra una IA local útil, privada y controlada.

El proyecto une interfaz, modelo local, API, RAG y agente dentro de una experiencia clara. La parte más importante no es solo que responda, sino que permite trabajar con contexto real, permisos visibles y una base preparada para seguir creciendo.

Privado Datos y memoria bajo control local.	Práctico Chat, agente y workspace en una misma interfaz.	Escalable Skills, perfiles y permisos preparados para crecer.
---	--	---

Documento preparado para presentar Vortex dentro del apartado de proyectos del portfolio.